CS429 Final Project

Roxanne Krause, Kurukulasuriya Leitan, Marnina Willard

May 8th, 2026

1 Map Abstraction

Each map the agent explores is represented by a black and white bitmap image, where the black pixels represent an obstacle, and white pixels represent a free pathway. These images, of varying size, are to be "over-approximated" and compressed in order to reduce the map for training. To elaborate, we look at a small section of an image, and if there is a single or more obstacle pixels, we make the single pixel in the corresponding compression an obstacle. This ensures that if the agent finds a path on the compression, it can assuredly find a path on the original map.

We will use the following steps to compress the map to a size of 40x40 if it is larger than said compression size. Let (n, m) be the neighborhood size (n rows, m columns) needed to translate the larger image section into a single pixel in the compression to achieve a 40x40 translation.

1. Extend the rows and columns to a number evenly divisible by 40. We shall extend using the values on the edges, maintaining any obstacles or paths in the extension.
2. Investigate every (n, m) sized distinct neighborhood in the original image. If any pixel in the neighborhood is an obstacle, mark the corresponding compression matrix pixel an obstacle.
3. Return the compression.

The extension used is done carefully to ensure the original image is still represented correctly and new paths are not mistakenly created. This makes the compression far simpler to conduct. Further, if any image is less than 40x40 already, we do not compress, given that it is already a sufficiently small size for training.

2 Building the Environment

The environment is built very simply to have a single interaction point for the agent. The agent provides the current state, action, and strategy, and the

environment returns the next state, reward, and if the agent has reached a terminal state. This environment controls the reward dispersal to the agent through the following strategies.

2.1 Reward Strategies

This section will detail the reward strategies used and implemented in the environment. Reward is only ever yielded to the agent upon request with the current state, action desired, and strategy to use.

2.1.1 S1

This strategy is a sparse, naive reward signal. The rules are as follows.

- If an obstacle is hit or the agent goes out of bounds: -100 reward.
- If the agent reaches the target: $+100$ reward.
- Any other normal step: 0 reward.

The agent receives no feedback between terminal events, meaning it must stumble upon the target through exploration before the Q-table can begin learning a useful policy. While simple, this serves as a useful baseline for comparing the effect of more informative reward signals.

2.1.2 S2

This strategy builds off S1 by adding a small penalty for each normal step taken. The rules are as follows.

- If an obstacle is hit or the agent goes out of bounds: -100 reward.
- If the agent reaches the target: $+100$ reward.
- Any other normal step: -1 reward.

The -1 step cost gives the agent feedback on every move, encouraging it to find the shortest path to the target rather than any path.

2.1.3 S3

This strategy uses the idea of rippling out a bait for the agent, and builds off the baseline of strategy S2. The rules are as follows.

- If an obstacle is hit or the agent goes out of bounds: -100 reward.
- If the agent reaches the target: $+100$ reward.
- Normal steps are -1 reward when outside of the ripple radius. Normal steps are $+1$ reward when inside of the ripple radius.

Given a target point, we use a Breadth-First style algorithm to "ripple" out from the target point, setting those state's rewards to +1. The rippling respects obstacles and boundaries, ensuring that if such a state is reached, that state is not explored further. This is done to ensure that we do not erroneously trap the agent into a dead-end without a real way to reach the target.

The strategy, by default, ripples out by $min(numRows, numCols)/3$ steps away from the target, $numRows$ and $numCols$ corresponding to the given map size. If any normal steps are not within that ripple radius or not reachable within the radius due to obstacles, the reward defaults to -1.

As an aside, the first attempt of the ripple strategy was to use descending rewards from the target: ripple away from the target with reward +100, each further step decreasing in 10 reward points (ie, 90, 80, ..., 10, 0). This introduced a phenomenon known as "reward hacking", in which the agent circles around the target, reaping the dense intermediate rewards rather than ever going for the largest target reward. The decision to make the baiting a low constant value was due to witnessing this issue firsthand. Empirically, rippling out half the length/width of the map also caused a high level reward hacking to occur. Through experimentation, rippling out by a factor of 1/3 appears to be enough bait without being so much to result in hacking. However, this hacking can still occur with different combinations of parameter balances, as we will discuss further in this report.

3 Building an Agent

Our agents generally have the baseline functionality of

- Creating, updating, and maintaining a Q Table (3D matrix)
- Interacting with the environment by requesting the next state and reward given an action.
- Training on a specific map, learning its environment. Additionally, testing on the same map given the Q Table policy.

Therefore, all agents begin with, at minimum:

- A Q Table, initialized with all 0 values, in which $Q[x][y][a]$ corresponds to the current policy for position (x, y) on the map and action taken a.
- A Q Table update method to override with the specific SARSA and Q-Learn update rules.
- An interaction wrapper around the environment to ask for next state and reward.
- A training method to override with the specific policy update iteration for SARSA or Q-Learn. Further, a testing method to utilize the final Q Table in path finding from a specified start point.

4 SARSA Process

For Task 4, we implemented a SARSA agent to learn a feedback policy for navigating the abstracted 2-D maps. Similar to the Q-Learning agent, each position in the map is treated as a state (x, y), and the agent can choose one of four actions: left, right, up, or down. The learned policy is stored in a 3-D Q-table, where the first dimension is the x-coordinate, the second dimension is the y-coordinate, and the third dimension represents the action:

$$Q[x][y][a]$$

During training, we iterate using the following steps to update the policy table based on actions and rewards from the environment.

```
for each episode:
    choose an initial state (x, y).
    choose an action A from (x, y), making an epsilon-greedy choice.

    for each step of this episode:
        take A, observe reward r and next state (x', y').

        if (x', y') is target or obstacle/out of bounds:
            update Q Table using SARSA terminal update.
            end the episode.

        choose next action A' from (x', y'), making an epsilon-greedy choice.
        update Q Table using SARSA update.
        (x, y) = (x', y').
        A = A'.
```

The main difference between SARSA and Q-Learning is the update rule. Q-Learning uses the maximum possible value from the next state, while SARSA uses the actual next action chosen by the current policy. Because of this, SARSA is considered an on-policy algorithm.

The updating of the Q-table for SARSA is as follows. Let the hyperparameters α and γ stand for the learning rate and the discount rate, respectively.

$$Q[x][y][A] \leftarrow Q[x][y][A] + \alpha\left[r + \gamma Q[x'][y'][A'] - Q[x][y][A]\right]$$

Here, (x, y) is the current state, A is the current action, r is the reward returned by the environment, (x', y') is the next state, and A' is the next action chosen from the next state. If the agent reaches a terminal state, meaning it reaches the target or hits an obstacle or boundary, then there is no next action to use. In this case, the update becomes:

$$Q[x][y][A] \leftarrow Q[x][y][A] + \alpha\left[r - Q[x][y][A]\right]$$

This terminal update is still performed before ending the episode so that the agent learns from both successful target states and failed crash states.

Unless specifically mentioned in any section, the unique strategies used for this agent are as follows.

- ϵ, for the ϵ-greedy choice of action, is kept constant.
- α and γ likewise remain constant.
- A random initial point is chosen from the free points on the map at the start of each episode.
- During action selection, the agent chooses from actions that stay within the map and do not immediately enter an obstacle.
- Reward shaping is turned off in the final version, so the agent uses the reward returned by the selected environment strategy.

For the final SARSA tests, we used Strategy S1. The implementation used the same general map, environment, and plotting structure as Q-Learning, but replaced the Q-Learning update with the SARSA update. The following is an example training and testing of SARSA on all four maps with the parameters:

- Learning Rate: 0.5
- Epsilon: 0
- Discount: 0.5
- Episodes: 5000
- Steps: 1000
- Strategy: S1

The target position was chosen as the bottom-right reachable point of each map. If the bottom-right corner was not a free cell, the closest free cell to the bottom-right corner was used instead. Similarly, the start position was chosen from the closest free cell to the preferred start point. The final learned path was plotted for each map, showing the start point, target point, transit points, and final path taken by the agent.

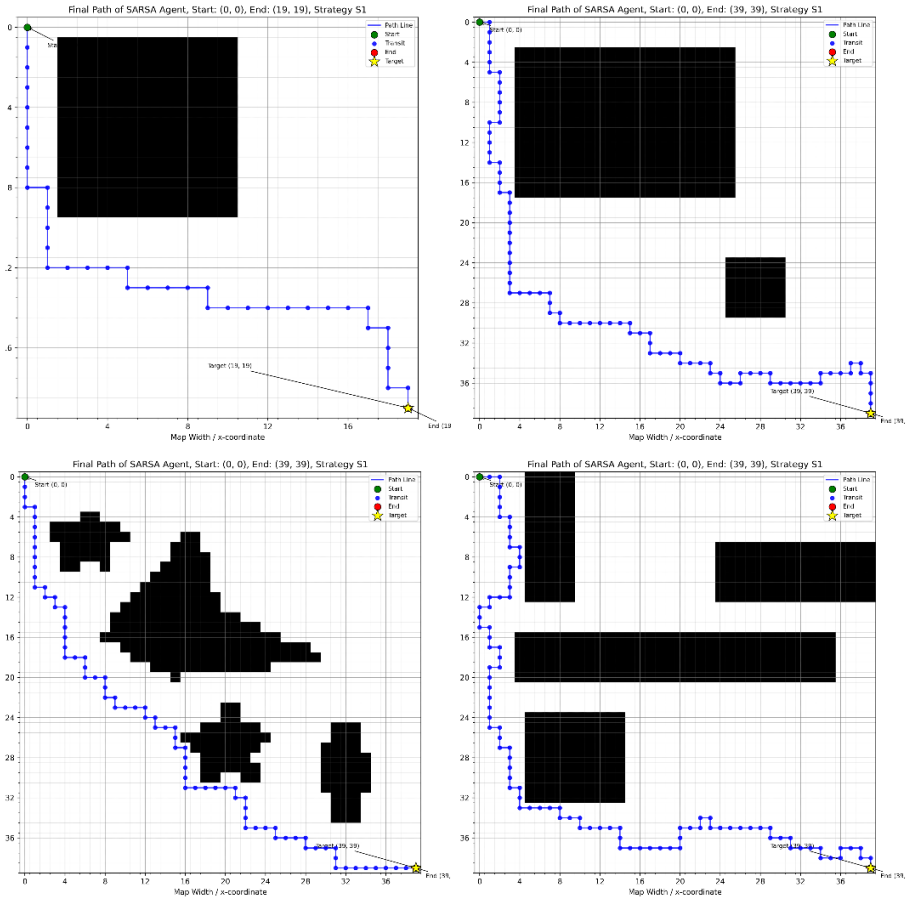

Figure 1: SARSA Maps

5 Q-Learning Process

For Task 5, we implemented a Q-Learn agent to learn a feedback policy for navigating the abstracted 2-D maps. To reiterate, each position in the map is treated as a state (x, y), and the agent can choose one of four actions: left, right, up, or down. The learned policy is stored in a 3-D Q-table, where the first dimension is the x-coordinate, the second dimension is the y-coordinate, and the third dimension represents the action:

$$Q[x][y][a]$$

During training, we iterate using the following steps to update the policy table based on actions and rewards from the environment.

```
for each episode:
  choose an initial state (x, y).

  for each step of this episode:
    choose an action A from (x, y), making an epsilon-greedy choice.
    take A, observe reward r and next state (x', y').
    update Q Table using Q-Learn update.
    if (x', y') is target or obstacle/out of bounds: end the episode.
    (x, y) = (x', y').
```

The updating of the Q Table for Q-Learn is as follows. Because Q-Learn uses the optimal policy for the next state, this is considered an off-policy method. Let the parameters α and γ stand for the learning rate and the discount rate respectively, while r is the reward.

$$Q[x][y][A] \leftarrow Q[x][y][A] + \alpha[r + (\gamma * max_a Q[x'][y'][a]) - Q[x][y][A]]$$

Unless specifically mentioned in any section, the unique strategies used for this agent are as follows.

- ϵ, for the ϵ-greedy choice of action, is kept constant.
- α and γ likewise remain constant.
- A random initial point is chosen from the free points on the map on the start of each episode.

The following is an example training and testing of Q-Learn on all four maps with the parameters:

- Learning Rate: 0.5
- Epsilon: 0
- Discount: 0.5
- Episodes: 5000
- Steps: 1000
- Strategy: S1

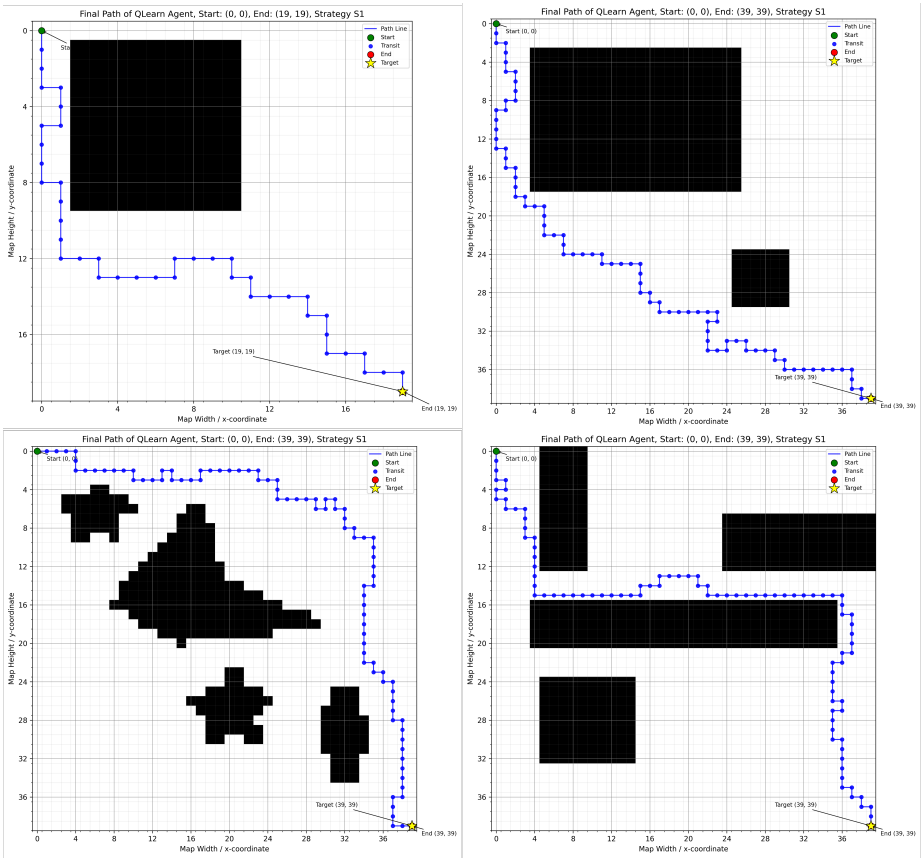

Figure 2: Q-Learn: Map 1

6 Performance Evaluation

In this section, we will explore the map complexity, effects of exploration and discount rate, and rewards stratgies on the learning process. For all of the following data collected, a consistent learning rate of 0.5 was used for both agents. Further, steps in both training episodes and testing rounds was capped to 1000.

6.1 Map Complexity

Both agents were trained on all four maps using fixed hyperparameters ($\varepsilon = 0.2$, $\gamma = 0.95$, Strategy S1) to isolate the effect of map structure on learning performance. On Map 1, the simplest map, both SARSA and Q-Learning achieved perfect accuracy (100%) with identical average path lengths of 17.9 steps, confirming that a small, open map presents no meaningful challenge to either

method. Performance begins to diverge on more complex maps. On Map 2, SARSA attained 99.1% accuracy with shorter average paths (39.0 steps) compared to Q-Learning's higher accuracy (99.8%) but longer paths (66.5 steps), suggesting Q-Learning finds the target from more starting positions but via less direct routes. On Map 3, Q-Learning reached 100% accuracy while SARSA fell to 86.3%, yet SARSA's successful paths were again shorter (39.8 vs. 52.1 steps). Map 4 produced the most pronounced contrast: SARSA achieved 94.5% accuracy with a compact average path of 42.5 steps, whereas Q-Learning reached only 78.9% and produced very long paths averaging 171.6 steps. This pattern suggests that SARSA's on-policy updates, which learn from the same exploratory actions taken during training, yield a more conservative and reliable policy on complex maps. Q-Learning's off-policy updates can overestimate reachable rewards in maze-like environments, leading to longer or failing trajectories.

Map	**Agent**	**Time (s)**	**Episodes**	**Accuracy (%)**	**Avg Path**
map1	SARSA	5.03	5000	100.0	17.9
map1	QLearn	3.81	5000	100.0	17.9
map2	SARSA	21.90	5000	99.1	39.0
map2	QLearn	14.95	5000	99.8	66.5
map3	SARSA	25.72	5000	86.3	39.8
map3	QLearn	16.29	5000	100.0	52.1
map4	SARSA	26.18	5000	94.5	42.5
map4	QLearn	14.87	5000	78.9	171.6

Table 1: Map complexity comparison. Both agents trained with $\varepsilon = 0.2$, $\gamma = 0.95$, strategy S1, 5000 episodes.

6.2 Exploration Rate

Experiment 2 fixed $\gamma = 0.5$ and Strategy S1 on Map 4, varying $\varepsilon \in \{0, 0.5, 1\}$. For SARSA, $\varepsilon = 0$ (pure exploitation) delivered the best overall result: 99.8% accuracy with an average path of 45.5 steps. Increasing exploration to $\varepsilon = 0.5$ produced a marginally lower accuracy (99.7%) with slightly shorter paths (39.4 steps), a negligible trade-off. Full exploration at $\varepsilon = 1$ degraded SARSA noticeably to 95.0% accuracy, and tripled training time (137.8 s) because the agent wastes episodes on random walks rather than reinforcing good paths. Q-Learning showed the opposite trend: $\varepsilon = 0$ yielded perfect accuracy (100%) and was the clear winner for that agent, while $\varepsilon = 0.5$ and $\varepsilon = 1$ both dropped to around 73–75% with extremely long average paths (160+ steps). This is consistent with Q-Learning's off-policy nature: because it always updates toward the greedy maximum regardless of the action taken, it benefits most when the policy being executed is also greedy. Based on these results, $\epsilon = 0$, $\gamma = 0.5$ is selected as the best combination for these results.

Agent	ε	Time (s)	Episodes	Accuracy (%)	Avg Path
SARSA	0	24.48	5000	99.8	45.5
QLearn	0	21.38	5000	100.0	46.4
SARSA	0.5	27.59	5000	99.7	39.4
QLearn	0.5	13.33	5000	73.4	178.3
SARSA	1	78.09	5000	95.0	39.3
QLearn	1	13.55	5000	75.7	162.7

Table 2: Exploration rate comparison on Map 4. Both agents trained with $\gamma = 0.5$, strategy S1, 5000 episodes.

6.3 Discount Value

Experiment 3 fixed $\varepsilon = 0.5$ and Strategy S1 on Map 4, varying $\gamma \in \{0.1, 0.5, 1.0\}$. For SARSA, $\gamma = 0.1$ produced the highest accuracy (100%) with paths averaging 39.9 steps and a reasonable training time of 43.7 s. At $\gamma = 0.5$ SARSA remained strong (99.7%), but $\gamma = 1.0$ caused a steep drop to only 53.6% accuracy — an undiscounted agent over-weights future rewards and can get trapped chasing distant targets in cycles. For Q-Learning, low discount ($\gamma = 0.1$) was again best at 75.8%, though all three settings hovered between 37–76%, confirming that Q-Learning struggles on this map regardless of discount. $\gamma = 1.0$ was particularly damaging for Q-Learning, dropping accuracy to 37.1% with an enormous average path of 739.7 steps. A low discount value effectively makes the agent more short-sighted, rewarding reaching the target quickly and penalizing long detours, which aligns well with a navigation task where the goal is to arrive efficiently. Based on these results, $\epsilon = 0.5$, $\gamma = 0.1$ is selected as the best combination for these results.

Agent	γ	Time (s)	Episodes	Accuracy (%)	Avg Path
SARSA	0.1	28.28	5000	100.0	39.9
QLearn	0.1	14.20	5000	75.8	175.6
SARSA	0.5	29.31	5000	99.7	39.4
QLearn	0.5	13.92	5000	73.4	178.3
SARSA	1	30.19	5000	53.6	43.0
QLearn	1	17.77	5000	37.1	739.7

Table 3: Discount value comparison on Map 4. Both agents trained with $\epsilon = 0.5$, strategy S1, 5000 episodes.

6.4 Reward Strategy

From Experiments 2 and 3, we considered the combinations ($\epsilon = 0$, $\gamma = 0.5$) and ($\epsilon = 0.5$, $\gamma = 0.1$) to provide the best performance metrics.

Agent	ε	γ	Time (s)	Episodes	Accuracy (%)	Avg Path
SARSA	0	0.5	24.48	5000	99.8	45.5
QLearn	0	0.5	21.38	5000	100.0	46.4
SARSA	0.5	0.1	28.28	5000	100.0	39.9
QLearn	0.5	0.1	14.20	5000	75.8	175.6

Table 4: Best results on Map 4, following strategy S1, 5000 episodes.

Observing both of the potential combinations, ($\epsilon = 0$, $\gamma = 0.5$) appears to have the better performance metrics. The time to train for either option is approximately the same. However, the accuracy and path length of Q-Learn highlight the largest difference, showcasing a that ($\epsilon = 0$, $\gamma = 0.5$) performs better with strategy S1 for both agents, avoiding an aimless wondering.

Experiment 4 applied the best hyperparameters from Experiments 2 and 3 ($\varepsilon = 0$, $\gamma = 0.5$) on Map 4 and compared the three reward strategies. Strategy S1 (sparse: $+100$ at goal, -100 on collision, 0 otherwise) performed best for both agents: SARSA reached 99.8% and Q-Learning reached 100% accuracy, with compact paths around 45-46 steps. Strategy S2 added a -1 step penalty to encourage shorter paths, but paradoxically hurt both agents significantly — SARSA fell to 76.2% accuracy and Q-Learning to 88.2%. The per-step penalty causes the agent to avoid long corridors even when they are the only route, collapsing the policy on a maze-like map. Strategy S3 attempted to guide the agent with a BFS ripple of positive rewards radiating from the target, but both agents essentially failed: SARSA reached an astounding 0.0% and Q-Learning only 0.5%, with near-trivial average path lengths indicating the agents almost never found the goal. The ripple reward inadvertently creates local optima near the target boundary that trap both agents. The conclusion is that for this navigation task, a simple sparse reward (S1) provides the clearest learning signal without introducing unintended gradient effects, making it the recommended strategy overall.

Agent	Strategy	Time (s)	Episodes	Accuracy (%)	Avg Path
SARSA	S1	85.63	5000	99.8	45.5
QLearn	S1	36.1	5000	100.0	46.4
SARSA	S2	31.02	5000	76.2	32.7
QLearn	S2	27.93	5000	88.2	50.0
SARSA	S3	332.5	5000	0.0	N/A
QLearn	S3	112.18	5000	0.5	3.2

Table 5: Reward strategy comparison on Map 4. Both agents trained with $\varepsilon = 0$, $\gamma = 0.5$, 5000 episodes.

6.4.1 Additional Discussion

Given the resulting best balance of exploration rate and weight on using learned policy, we can see and verify through further experimentation the importance

of a carefully designed reward system and parameter tuning. We will try a combination of ($\varepsilon = 0.2$, $\gamma = 0.95$) on Map 4 with all strategies.

Agent	Strategy	Time (s)	Episodes	Accuracy (%)	Avg Path
SARSA	S1	84.36	5000	94.5	42.5
QLearn	S1	107.9	5000	78.9	171.6
SARSA	S2	155.77	5000	93.1	37.8
QLearn	S2	108.72	5000	71.0	30.8
SARSA	S3	108.01	5000	94.9	38.8
QLearn	S3	83.15	5000	83.9	36.9

Table 6: Reward strategy comparison on Map 4. Both agents trained with $\varepsilon = 0.2$, $\gamma = 0.95$, 5000 episodes.

Although this combination clearly has a performance drop for strategy S1, we see an overall marked improvement in path-finding metrics for strategies S2 (for SARSA) and S3 (for both). By altering the parameters, we can find a better accuracy in a given strategy. However, S1 with our best naive choice of $\varepsilon = 0$, $\gamma = 0.5$ still yields a higher accuracy, path length, and training time than all of the above trials. Therefore, not only is the tuning of parameters of utmost importance, but also carefully designing the reward strategy is a crucial piece of the reinforcement learning process. Despite believing certain rewards would have benefits during design, there is no guarantee that the additional noise will yield higher overall performance.

7 Contribution

Kurukulasuriya Leitan

- Environment
- Performance Evaluation

Marnina Willard

- SARSA Model
- Path Plotting

Roxanne Krause

- Map Compression
- Reward Strategy S3
- Baseline Agent
- QLearn Agent
- Performance Evaluation: Additional Discussion